

Práctica Entregable 2: Automatización y Transformación de Flujos de Datos en la Nube



Daniel Moreno López, 4ºCurso

ULPGC, EII, GII

Computación en la Nube

Fecha entrega: 09/01/2026

Índice

1. Introducción.....	3
2. Desarrollo de las actividades.....	5
2.1. Configuración del bucket S3.....	5
2.2. Implementación del productor de datos.....	7
2.3. Configuración del consumidor (Kinesis Firehose).....	8
2.4. Configuración de AWS Glue.....	11
2.5. Análisis de datos con Amazon Athena.....	14
3. Diagrama del flujo de datos.....	16
4. Presupuesto y estimación de costes.....	18
5. Conclusiones.....	20
6. Referencias y Bibliografía.....	22
Anexo A: Código Fuente y Scripts.....	23
A.1. Generador de Datos.....	23
A.2. Transformación en Tránsito.....	25
A.3. Procesamiento ETL Diario.....	27
A.4. Procesamiento ETL Mensual.....	29
A.5. Script de Despliegue de Infraestructura (AWS CLI).....	31
Anexo B: Reporte de Costes.....	34
Anexo C: Uso de la Inteligencia Artificial.....	37
Anexo D: Enlace a Github.....	38

1. Introducción

El objetivo principal de esta práctica es diseñar e implementar una arquitectura de datos escalable y serverless en AWS para la ingesta, procesamiento y análisis de información.

Para lograr este objetivo, se ha desplegado un pipeline de ingeniería de datos completo que simula la recepción de registros de conectividad aérea y los procesa para obtener métricas agregadas, como el volumen de enlaces por país y la evolución temporal del tráfico.

La solución se apoya en los siguientes servicios gestionados de AWS, que permiten desacoplar el almacenamiento del cómputo y optimizar costes:

- **Amazon S3 (Simple Storage Service):** Actúa como el Data Lake centralizado del proyecto. Se ha estructurado en diferentes capas para garantizar la durabilidad, disponibilidad y escalabilidad del almacenamiento de los archivos JSON y Parquet generados.
- **Amazon Kinesis Data Streams:** Servicio de streaming de datos en tiempo real que permite la ingesta masiva de registros desde el productor con baja latencia, actuando como el punto de entrada de la arquitectura.
- **Amazon Kinesis Data Firehose:** Servicio encargado de consumir los datos del stream, aplicar una transformación ligera mediante una función AWS Lambda y entregarlos de manera persistente en el bucket de S3.
- **AWS Glue:** Servicio de integración de datos serverless utilizado en dos vertientes:
 - **Glue Crawler:** Para descubrir automáticamente el esquema de los datos almacenados y catalogarlos.
 - **Glue ETL Jobs:** Para ejecutar trabajos de procesamiento distribuido que limpian, agregan y transforman los datos crudos en información analítica valiosa.

Esta arquitectura permite transformar datos desestructurados o semi-estructurados en información de negocio lista para ser consultada mediante herramientas SQL como Amazon Athena, cumpliendo con los principios modernos de ingeniería de datos en la nube.

2. Desarrollo de las actividades

En este apartado se detalla el proceso de implementación de la arquitectura de datos, desde la creación de la infraestructura de almacenamiento hasta la automatización de los procesos ETL.

2.1. Configuración del bucket S3

Como paso inicial, se provisiona un bucket en **Amazon S3** para servir como Data Lake centralizado del proyecto. La configuración se realizó siguiendo una estrategia de particionado lógico mediante carpetas para separar las diferentes etapas del ciclo de vida del dato.

Se creó la siguiente estructura de directorios como también se comprueba en la Figura1:

- **/raw:** Destino de los datos crudos ingestados directamente desde Kinesis Firehose.
- **/processed:** Almacenamiento de los datos procesados y refinados tras la ejecución de los trabajos ETL.
- **/scripts:** Repositorio para almacenar los scripts de Python utilizados por los Jobs de AWS Glue.
- **/errors:** Se ha configurado esta carpeta específica para capturar aquellos registros que no pudieron ser procesados correctamente por Kinesis Firehose o la función Lambda de transformación. Esto actúa como una "Dead Letter Queue" (DLQ) a nivel de almacenamiento, permitiendo auditar y depurar datos corruptos sin detener el flujo principal.
- **/config y /logs:** Para configuraciones adicionales y registros de auditoría.

Esta estructura permite desacoplar la ingesta del procesamiento, manteniendo una copia inmutable de los datos originales en raw y ofreciendo datos optimizados en processed para reducir costes y tiempos de consulta en Athena.

La creación de esta infraestructura no se realizó de forma manual, sino mediante la interfaz de línea de comandos, AWS CLI. Esto asegura que el entorno sea reproducible y evita errores humanos.

Se utilizó el comando `aws s3 mb` para aprovisionar el bucket y `aws s3api put-object` para generar la estructura de carpetas:

```
# Crear el bucket
aws s3 mb s3://$env:BUCKET_NAME

# Crear carpetas (objetos vacíos con / al final)
aws s3api put-object --bucket $env:BUCKET_NAME --key raw/
aws s3api put-object --bucket $env:BUCKET_NAME --key processed/
aws s3api put-object --bucket $env:BUCKET_NAME --key config/
aws s3api put-object --bucket $env:BUCKET_NAME --key scripts/
aws s3api put-object --bucket $env:BUCKET_NAME --key queries/
aws s3api put-object --bucket $env:BUCKET_NAME --key errors/
```

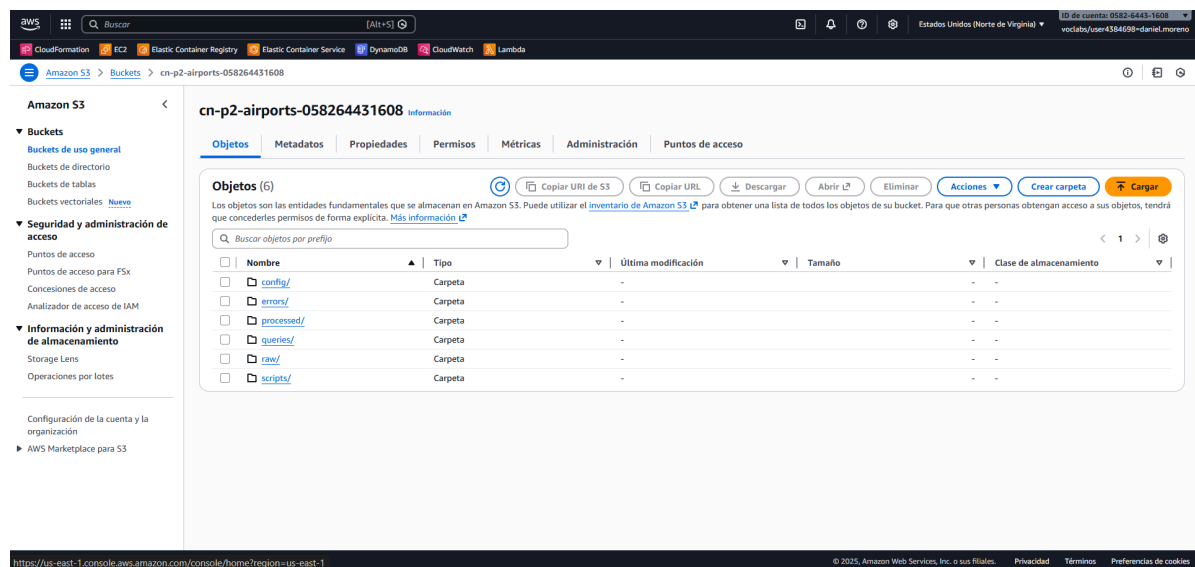


Figura1: Organización de carpetas S3

2.2. Implementación del productor de datos

Para simular la generación de datos de tráfico aéreo en tiempo real, se ha desarrollado e implementado un script en Python llamado `kinesisDML.py`. Este script actúa como un productor de datos, inyectando registros continuamente.

El script utiliza la librería `boto3` para conectarse al servicio Amazon Kinesis Data Streams. Lee un conjunto de datos base y envía cada registro al stream denominado `airports-stream`.

Cada registro enviado contiene información sobre el país de origen y el número de conexiones (`links_count`). Para simular un flujo de datos realista y evitar la saturación, se ha configurado una frecuencia de envío controlada mediante un retardo (`time.sleep`) entre iteraciones.

Debido a la extensión del código fuente desarrollado para este componente, el script completo se encuentra documentado en el Anexo.

Como evidencia del correcto funcionamiento del productor, en la Figura 2 se muestra una captura de la terminal durante la ejecución del script, donde se observan los mensajes de confirmación de envío de los registros al stream.

```
PS C:\Users\danie\Desktop\CN_P2> python .\kinesisDML.py
2025-12-30 12:45:21.118 INFO __main__:run_producer:22 - Iniciando transmisión al stream: airports-stream...
2025-12-30 12:45:21.914 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:21.914 INFO __main__:run_producer:43 - Enviado: Hartsfield Jackson Atlanta Intl (United States)
2025-12-30 12:45:22.076 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.076 INFO __main__:run_producer:43 - Enviado: Chicago Ohare Intl (United States)
2025-12-30 12:45:22.273 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.274 INFO __main__:run_producer:43 - Enviado: Capital Intl (China)
2025-12-30 12:45:22.431 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.432 INFO __main__:run_producer:43 - Enviado: Heathrow (United Kingdom)
2025-12-30 12:45:22.593 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.595 INFO __main__:run_producer:43 - Enviado: Charles De Gaulle (France)
2025-12-30 12:45:22.759 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.760 INFO __main__:run_producer:43 - Enviado: Los Angeles Intl (United States)
2025-12-30 12:45:22.915 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:22.915 INFO __main__:run_producer:43 - Enviado: Frankfurt Main (Germany)
2025-12-30 12:45:23.070 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.071 INFO __main__:run_producer:43 - Enviado: Dallas Fort Worth Intl (United States)
2025-12-30 12:45:23.242 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.243 INFO __main__:run_producer:43 - Enviado: John F Kennedy Intl (United States)
2025-12-30 12:45:23.398 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.399 INFO __main__:run_producer:43 - Enviado: Schiphol (Netherlands)
2025-12-30 12:45:23.558 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.558 INFO __main__:run_producer:43 - Enviado: Pudong (China)
2025-12-30 12:45:23.711 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.711 INFO __main__:run_producer:43 - Enviado: Changi Intl (Singapore)
2025-12-30 12:45:23.867 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:23.867 INFO __main__:run_producer:43 - Enviado: Barcelona (Spain)
2025-12-30 12:45:24.039 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:24.039 INFO __main__:run_producer:43 - Enviado: Incheon Intl (South Korea)
2025-12-30 12:45:24.210 INFO __main__:run_producer:42 - Registro enviado al shard shardId-000000000000
2025-12-30 12:45:24.210 INFO __main__:run_producer:43 - Enviado: Denver Intl (United States)
```

Figura2: Ejecución `kinesisDML.py`

2.3. Configuración del consumidor (Kinesis Firehose)

Para gestionar la ingesta y persistencia de los datos en el Data Lake, se configuró un Amazon Kinesis Data Firehose (airports-delivery-stream) que actúa como consumidor directo del Kinesis Data Stream implementado en el apartado anterior.

La función principal de este componente es capturar los registros de alta velocidad, agruparlos y depositarlos de manera organizada en Amazon S3.

Se definió el bucket creado previamente como destino final. Para mantener la organización del Data Lake, se configuraron dos prefijos clave:

- **Prefijo de datos:** raw/. Firehose añade automáticamente una estructura de carpetas particionada por tiempo (YYYY/MM/DD/HH) bajo este prefijo, lo que facilita la navegación y optimiza futuras consultas.
- **Prefijo de error:** errors/. Se habilitó este prefijo para capturar y aislar automáticamente cualquier registro que falle durante el procesamiento o la entrega, implementando un mecanismo de seguridad para evitar la pérdida de datos.

Dado que los registros JSON en bruto llegan concatenados, es necesario procesarlos antes de su almacenamiento para que herramientas como AWS Glue puedan interpretarlos correctamente. Para ello, se habilitó la característica de Transformación de Datos de Firehose, vinculándola a una función AWS Lambda desarrollada en Python (firehoseDML.py).

```
aws lambda create-function --function-name airports-firehose-lambda
--runtime python3.12 --role $env:ROLE_ARN --handler
firehoseDML.lambda_handler --zip-file fileb://firehoseDML.zip --timeout 60
--memory-size 128
```

El código Python desplegado aplica una lógica de enriquecimiento y formateo. A continuación, se muestra el fragmento clave de dicha transformación:


```

# 1. Enriquecimiento: Obtener fecha actual en UTC
processing_time = datetime.datetime.now(datetime.timezone.utc)

data_json['processed_at'] = processing_time.isoformat()

# 2. Formateo: Convertir a JSON y añadir salto de línea
payload_transformed = json.dumps(data_json) + '\n'

# 3. Codificación final en Base64 para devolver a Firehose
output_record = {
    'recordId': record['recordId'],
    'result': 'Ok',
    'data':
base64.b64encode(payload_transformed.encode('utf-8')).decode('utf-8')
}

```

Esta función aplica las siguientes operaciones sobre cada registro:

1. Enriquecimiento: Añade el campo `processed_at` con la marca de tiempo UTC exacta del momento de la ingesta.
2. Formateo: Inserta un salto de línea (`\n`) al final de cada objeto JSON. Esto es indispensable para generar archivos JSON Lines (NDJSON) válidos, donde cada línea corresponde a un registro independiente.

Una vez lista la Lambda, se procedió a crear el Firehose conectando el origen (Kinesis Stream) con el destino (S3) y vinculando la transformación. Se definieron los prefijos `raw/` para datos y `errors/` para fallos, además de un buffer de 1 MB o 60 segundos.

El comando de infraestructura utilizado fue:

```
aws firehose create-delivery-stream --delivery-stream-name
airports-delivery-stream --delivery-stream-type KinesisStreamAsSource
--kinesis-stream-source-configuration ("KinesisStreamARN=arn:aws:kinesis:" +
$env:AWS_REGION + ":" + $env:ACCOUNT_ID + ":stream/airports-stream,RoleARN="
+ $env:ROLE_ARN) --extended-s3-destination-configuration
('{"BucketARN":"arn:aws:s3::' + $env:BUCKET_NAME + '\",\"RoleARN\":\"' +
$env:ROLE_ARN +
'\",\"Prefix\":\"raw/\", \"ErrorOutputPrefix\":\"errors/\", \"BufferingHints\"
:{\"SizeInMBs\":\"1\", \"IntervalInSeconds\":\"60\"}, \"ProcessingConfiguration\":{\"E
nabled\":true, \"Processors\": [{\"Type\":\"Lambda\", \"Parameters\": [{\"Parame
terName\":\"LambdaArn\", \"ParameterValue\":\"' + $env:LAMBDA_ARN +
'\", {\"ParameterName\":\"BufferSizeInMBs\", \"ParameterValue\":\"1\"}, {\"Par
ameterName\":\"BufferIntervalInSeconds\", \"ParameterValue\":\"60\"}]}]}]')

```

En la Figura 3 se muestra la configuración del Delivery Stream en la consola de AWS, destacando la transformación habilitada y el destino S3.

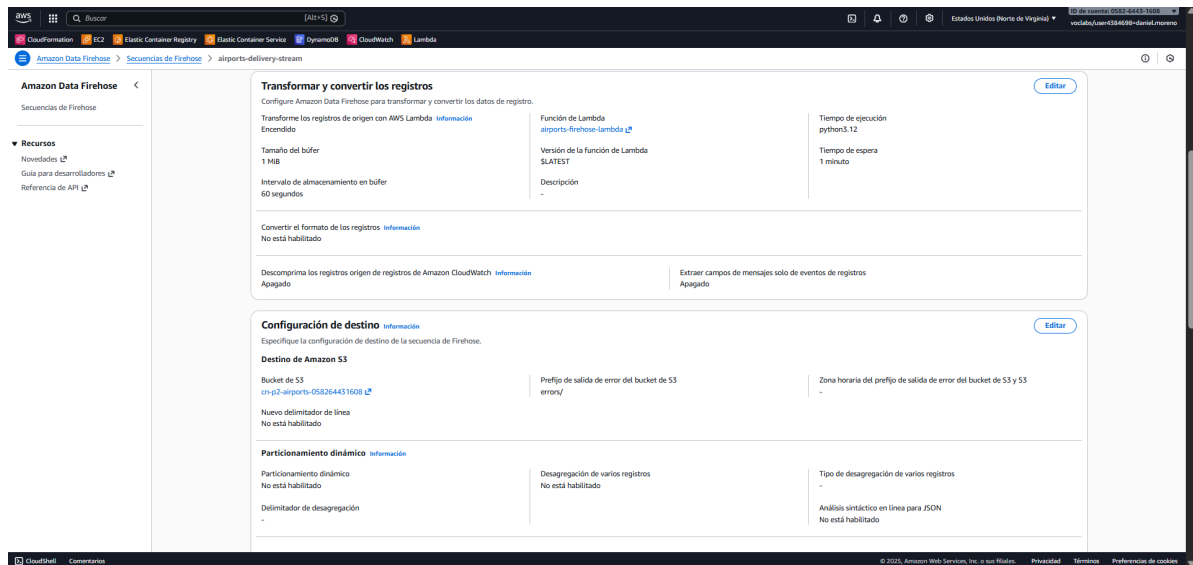


Figura3: Configuración de Kinesis Firehose

2.4. Configuración de AWS Glue

Para la gestión del catálogo de datos y la ejecución de los procesos ETL, se utilizaron los servicios de AWS Glue. La configuración de estos recursos se realizó íntegramente mediante AWS CLI para garantizar la reproducibilidad del entorno..

En primer lugar, se creó una base de datos lógica llamada `airports_db` para alojar los metadatos. Posteriormente, se configuró y lanzó un Crawler (`airports-raw-crawler`) apuntando a la ruta `raw/` del bucket S3. Su función es recorrer los datos crudos, inferir el esquema y crear la tabla correspondiente.

Los comandos ejecutados para este despliegue fueron:

```
#Base de datos

aws glue create-database --database-input '{"Name": "airports_db"}'

#Crawler

aws glue create-crawler --name airports-raw-crawler --role $env:ROLE_ARN
--database-name airports_db --targets ('{"S3Targets": [{"Path": "s3://"
+ $env:BUCKET_NAME + '/raw/"}]}')

aws glue start-crawler --name airports-raw-crawler
```

Tras la finalización del crawler, se verificó en la consola que la tabla `raw` había sido creada correctamente en el catálogo como se ve en la Figura 4.

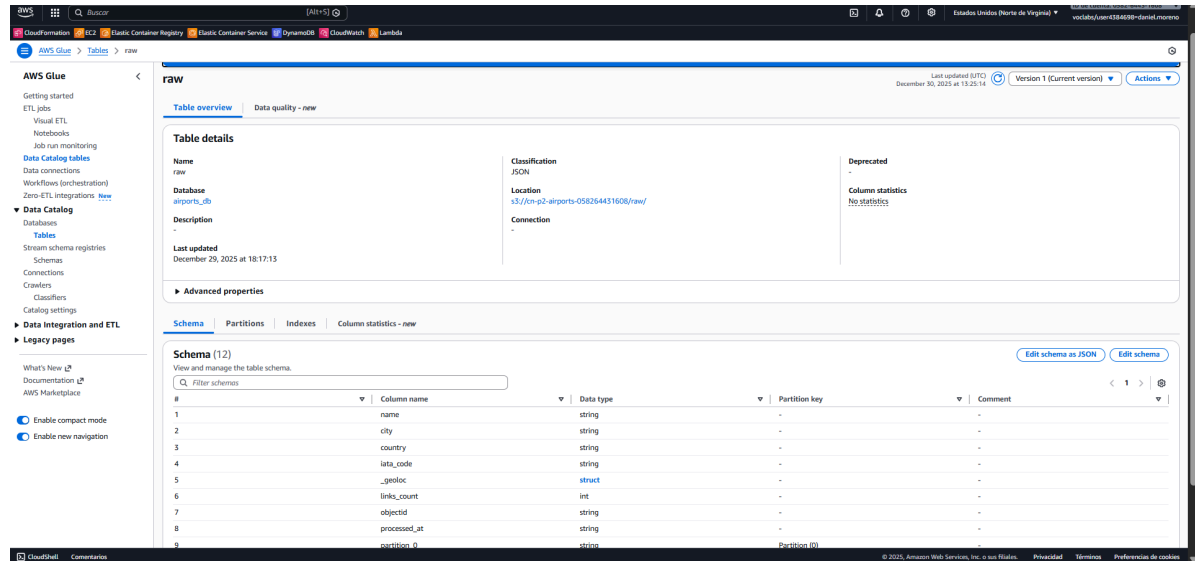


Figura4: Tabla raw

Se definieron dos trabajos de procesamiento distribuido utilizando Apache Spark sobre AWS Glue. Estos jobs transforman los datos crudos en información agregada.

- Job Diario (airports-daily-aggregation): Agrupa los datos por país y calcula los totales.
- Job Mensual (airports-monthly-aggregation): Agrupa por mes para análisis de tendencias.

Para la creación y puesta en marcha de estos jobs, se utilizó el siguiente script:

```
aws s3 cp airports_aggregation_daily.py s3://$env:BUCKET_NAME/scripts/
aws s3 cp airports_aggregation_monthly.py s3://$env:BUCKET_NAME/scripts/

$env:DATABASE="airports_db"
$env:TABLE_INPUT="raw"

aws glue create-job --name airports-daily-aggregation --role $env:ROLE_ARN
--glue-version "4.0" --number-of-workers 2 --worker-type "G.1X" --command
('{\"Name\": \"glueetl\", \"ScriptLocation\": \"s3://' + $env:BUCKET_NAME +
'/scripts/airports_aggregation_daily.py\", \"PythonVersion\": \"3\"}')
--default-arguments ('{\"--database\": \"' + $env:DATABASE + '\",
\"--table\": \"' + $env:TABLE_INPUT + '\", \"--output_path\": \"s3://' +
$env:BUCKET_NAME + '/processed/airports_daily/\",
\"--enable-continuous-cloudwatch-log\": \"true\"}')
```

```
aws glue create-job --name airports-monthly-aggregation --role $env:ROLE_ARN
```

```
--glue-version "4.0" --number-of-workers 2 --worker-type "G.1X" --command
('{\"Name\": \"glueetl\", \"ScriptLocation\": \"s3://' + $env:BUCKET_NAME +
'/scripts/airports_aggregation_monthly.py\", \"PythonVersion\": \"3\"}')
--default-arguments ('{\"--database\": \"' + $env:DATABASE + '\",
\"--table\": \"' + $env:TABLE + '\", \"--output_path\": \"s3://' +
$env:BUCKET_NAME + '/processed/airports_monthly/',
\"--enable-continuous-cloudwatch-log\": \"true\"}')
```

```
aws glue start-job-run --job-name airports-daily-aggregation
aws glue start-job-run --job-name airports-monthly-aggregation
```

Finalmente, ambos trabajos finalizaron con estado SUCCEEDED como se observa en la Figura 5, depositando los resultados en la carpeta processed/ del bucket S3.

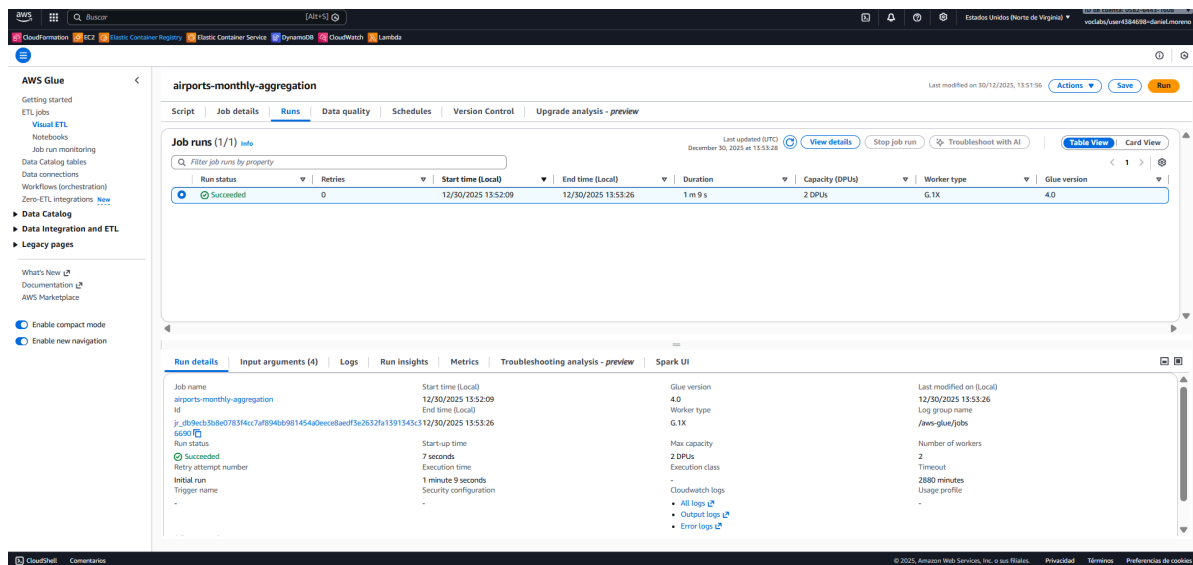


Figura5: Job ejecutado correctamente

2.5. Análisis de datos con Amazon Athena

Como paso final de la actividad, se validó la disponibilidad de los datos procesados utilizando Amazon Athena. Para que los archivos Parquet generados por los Jobs ETL fueran accesibles mediante SQL, fue necesario catalogarlos previamente en el Data Catalog.

Para ello, se automatizó la creación de un Crawler específico que monitoriza las carpetas de salida de ambos procesos ETL (daily y monthly). El siguiente comando despliega este recurso apuntando a múltiples rutas de S3 simultáneamente:

```
aws glue create-crawler --name airports-processed-crawler --role
$env:ROLE_ARN --database-name airports_db --targets ('{"S3Targets":
[{"Path": "s3://" + $env:BUCKET_NAME + "/processed/airports_daily/"},
{"Path": "s3://" + $env:BUCKET_NAME +
"/processed/airports_monthly/"}]'}')

aws glue start-crawler --name airports-processed-crawler
```

Una vez que el crawler actualizó el Catálogo de Datos registrando la tabla airports_daily, fue posible realizar consultas interactivas. Se ejecutó la siguiente sentencia SQL para obtener el ranking de tráfico aéreo:

```
SELECT
    country AS "País",
    cantidad_aeropuertos AS "Total Aeropuertos",
    total_enlaces AS "Total Enlaces Aéreos"
FROM "airports_db"."airports_daily"
ORDER BY total_enlaces DESC;
```

Como se observa en la Figura 6, la consulta devolvió los resultados esperados en menos de un segundo, confirmando el éxito del pipeline.

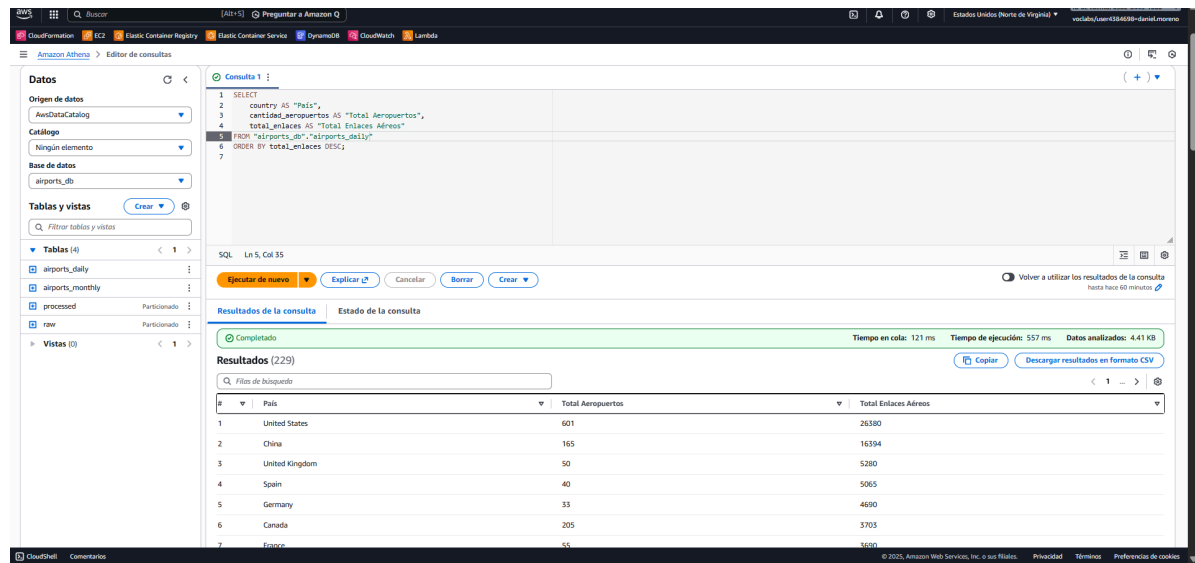


Figura 6: Consulta SQL en Amazon Athena mostrando los resultados agregados por país.

3. Diagrama del flujo de datos

Para facilitar la comprensión de la arquitectura implementada, se ha diseñado un diagrama que ilustra el ciclo de vida completo del dato, desde su generación en el cliente hasta su explotación analítica.

El flujo de datos, representado en la Figura 7, sigue la siguiente secuencia lógica:

1. **Generación:** El cliente externo ejecuta el script productor (`kinesisDML.py`), enviando registros JSON mediante la operación `PutRecord`.
2. **Ingesta:** Amazon Kinesis Data Streams recibe el flujo de datos en tiempo real, garantizando la durabilidad y orden de los registros.
3. **Buffer y Transformación:** Amazon Data Firehose consume los datos del stream. Antes de la persistencia, invoca a una función AWS Lambda (`firehoseDML.py`) que formatea y enriquece cada registro.
4. **Almacenamiento:** Los datos transformados se depositan en el bucket de Amazon S3 dentro de la carpeta `/raw`.
5. **Catalogación:** El AWS Glue Crawler analiza los archivos en S3 para inferir el esquema y actualizar la definición de la tabla en el Data Catalog (`airports_db`).
6. **Procesamiento ETL:** Los trabajos de AWS Glue leen los datos crudos, aplican las agregaciones y escriben los resultados optimizados en la carpeta `/processed`.
7. **Análisis:** Finalmente, Amazon Athena utiliza los metadatos del catálogo para lanzar consultas SQL directamente sobre los archivos procesados en S3.

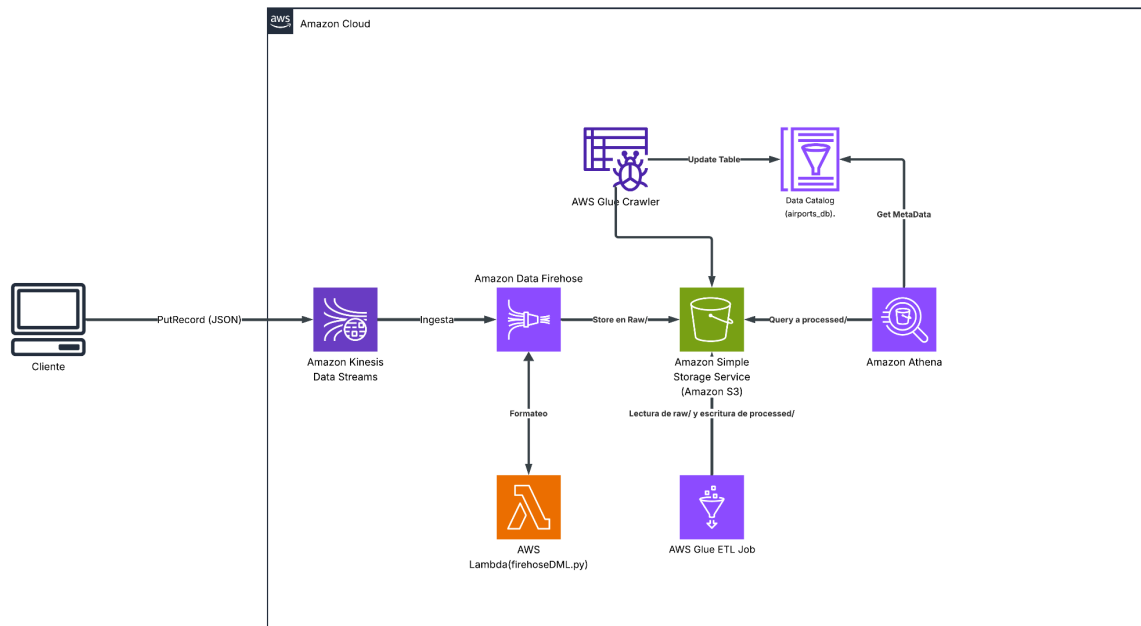


Figura 7: Arquitectura de flujo de datos implementada en AWS.

4. Presupuesto y estimación de costes

Para obtener una estimación fiel a la realidad, no se han utilizado los datos de la prueba, sino que se ha proyectado el sistema operando las 24 horas del día durante un mes completo bajo las siguientes condiciones de carga:

- Ingesta Continua: Flujo constante de 20 registros por segundo (aprox. 51 millones de eventos al mes).
- Retención: Almacenamiento histórico de 300 GB/mes.
- Procesamiento: Ejecución diaria de trabajos ETL para transformar y agregar esta información.

Utilizando la herramienta oficial AWS Pricing Calculator, se han obtenido los costes operativos que se detallan en la siguiente tabla. El informe completo con el desglose técnico de cada servicio se adjunta con la entrega de esta memoria.

Como se observa en los datos obtenidos en la Tabla 1, mantener esta arquitectura activa para monitorizar el tráfico aéreo global tendría un coste aproximado de 56,06 dólares al mes donde se destacan los siguientes puntos:

1. El uso de servicios como Lambda y Athena tiene un impacto marginal en la factura, demostrando que pagar solo por el uso real es altamente rentable frente a mantener servidores dedicados.
2. Los costes más elevados provienen de Kinesis y Glue, servicios necesarios para garantizar la capacidad de computación y el ancho de banda para procesar los flujos de datos sin latencia.
3. El coste anual proyectado de \$672.72 resulta competitivo para una solución de Big Data capaz de procesar más de 600 millones de registros al año con alta disponibilidad.

Tabla1: Estimación de precios

Servicio	Configuración Estimada	Coste Mensual (USD)	Coste Anual (USD)
Amazon Kinesis Data Streams	Ingesta en tiempo real (Stream aprovisionado).	\$22.64	\$271.68
AWS Glue	Procesamiento ETL (Spark) y Catalogación.	\$17.60	\$211.20
Amazon S3	Almacenamiento (300 GB) y solicitudes PUT/GET.	\$7.44	\$89.28
Amazon Data Firehose	Entrega de datos (20 reg/seg) a S3.	\$7.27	\$87.24
AWS Lambda	Transformación de eventos (Enriquecimiento).	\$0.82	\$9.84
Amazon Athena	Consultas SQL analíticas (Escaneo de datos).	\$0.29	\$3.48
TOTAL ESTIMADO	Infraestructura completa	\$56.06	\$672.72

5. Conclusiones

La realización de esta práctica ha permitido diseñar, desplegar y validar una arquitectura de ingeniería de datos completa en la nube de AWS, cumpliendo con los objetivos de escalabilidad, automatización y eficiencia de costes definidos inicialmente.

Tras la implementación de los servicios de ingesta, almacenamiento, procesamiento y análisis, se extraen las siguientes conclusiones principales:

1. La arquitectura implementada demuestra cómo el uso de servicios gestionados elimina la necesidad de aprovisionar y mantener servidores. Componentes como AWS Lambda y Kinesis Firehose han permitido desacoplar la lógica de negocio de la infraestructura, facilitando un escalado automático ante el flujo de datos simulado.
2. A diferencia de una configuración manual, el uso intensivo de AWS CLI y scripts en Python para el despliegue de recursos garantiza la reproducibilidad del entorno. Se ha comprobado que automatizar tareas, como la creación de Crawlers o la definición de Jobs ETL, reduce drásticamente los errores humanos y agiliza la puesta en producción.
3. Se ha evidenciado la diferencia crítica entre almacenar datos y hacerlos útiles. La transformación de archivos JSON crudos a formato columnar Parquet mediante AWS Glue, junto con su catalogación automática, ha sido determinante para permitir consultas SQL rápidas y eficientes en Amazon Athena, reduciendo tanto el tiempo de respuesta como el coste por escaneo.
4. El análisis presupuestario realizado confirma que la solución es altamente rentable. Con un coste estimado de \$56,06 USD mensuales para un entorno de producción que procesa más de 50 millones de registros al mes, la arquitectura valida la premisa de que el modelo de pago por uso de la nube democratiza el acceso a tecnologías de Big Data.

En definitiva, la práctica ha servido para comprender e implementar el flujo completo de la información en un entorno Cloud moderno, desde su generación en tiempo real hasta su

transformación en valor analítico para el negocio.

6. Referencias y Bibliografía

Amazon Web Services. (s.f.). Amazon Athena User Guide. Recuperado de

<https://docs.aws.amazon.com/athena/latest/ug/what-is.html>

Amazon Web Services. (s.f.). Amazon Kinesis Data Firehose Developer Guide. Recuperado de

de <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>

Amazon Web Services. (s.f.). Amazon Kinesis Data Streams Developer Guide. Recuperado de

<https://docs.aws.amazon.com/kinesis/latest/dev/introduction.html>

Amazon Web Services. (s.f.). AWS Command Line Interface User Guide. Recuperado de

<https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-welcome.html>

Amazon Web Services. (s.f.). AWS Glue Developer Guide. Recuperado de

<https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html>

Amazon Web Services. (s.f.). AWS Pricing Calculator. <https://calculator.aws/>

Amazon Web Services. (s.f.). Boto3 Documentation: AWS SDK for Python. Recuperado de

<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>

Axelcab. (s.f.). p5_aula [Repositorio de GitHub]. GitHub. https://github.com/Axelcab/p5_aula

Google. (s.f.). Gemini [Modelo de lenguaje grande]. Recuperado de

<https://gemini.google.com/>

OpenAI. (s.f.). ChatGPT [Modelo de lenguaje grande]. Recuperado de <https://chatgpt.com/>

Anexo A: Código Fuente y Scripts

En este anexo se recopila la totalidad del código fuente desarrollado para la implementación de la arquitectura de datos. Se incluyen los scripts de generación de datos, las funciones de transformación y los trabajos ETL de procesamiento masivo.

A.1. Generador de Datos

Archivo: kinesisDML.py

Este script simula la fuente de datos en tiempo real. Utiliza el SDK de AWS (boto3) para generar registros y enviarlos al flujo de datos Amazon Kinesis Data Streams mediante el método `put_record`.

```
import boto3
import json
import time
from loguru import logger
import datetime

STREAM_NAME = 'airports-stream'
REGION = 'us-east-1'
INPUT_FILE = 'airports.json'

kinesis = boto3.client('kinesis', region_name=REGION)

def load_data(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        return json.load(f)

def run_producer():
    data = load_data(INPUT_FILE)
    records_sent = 0

    logger.info(f"Iniciando transmisión al stream: {STREAM_NAME}...")

    for airport in data:
        try:
```

```
payload = airport

pais = airport.get('country', 'Unknown')

# Enviar a Kinesis
response = kinesis.put_record(
    StreamName=STREAM_NAME,
    Data=json.dumps(payload),
    PartitionKey=pais
)

records_sent += 1

# Ver qué aeropuerto se envía
logger.info(f"Registro enviado al shard {response['ShardId']}")
logger.info(f"Enviado: {airport.get('name')} ({pais})")

time.sleep(0.01)

except Exception as e:
    logger.error(f"Error enviando registro: {e}")

logger.info(f"Fin de la transmisión. Total registros enviados:
{records_sent}")

if __name__ == '__main__':
    run_producer()
```


A.2. Transformación en Tránsito

Archivo: firehoseDML.py

Código desplegado en AWS Lambda que actúa como función de transformación para Amazon Data Firehose. Su objetivo es procesar los registros en bruto antes de su persistencia, añadiendo saltos de línea y metadatos necesarios para asegurar que el formato JSON final almacenado en S3 sea válido y procesable.

```
import json
import base64
import datetime

def lambda_handler(event, context):
    output = []
    for record in event['records']:
        try:

            payload = base64.b64decode(record['data']).decode('utf-8')
            data_json = json.loads(payload)

            if 'country' not in data_json or not data_json['country']:
                output_record = {
                    'recordId': record['recordId'],
                    'result': 'Dropped',
                    'data': record['data']
                }
            else:

                processing_time =
datetime.datetime.now(datetime.timezone.utc)
                data_json['processed_at'] = processing_time.isoformat()

                payload_transformed = json.dumps(data_json) + '\n'

                output_record = {
                    'recordId': record['recordId'],
                    'result': 'Ok',
                    'data':
base64.b64encode(payload_transformed.encode('utf-8')).decode('utf-8')
                }
        
```

```
        output.append(output_record)

    except Exception as e:
        print(f"Error: {e}")
        output.append({
            'recordId': record['recordId'],
            'result': 'ProcessingFailed',
            'data': record['data']
        })

    return {'records': output}
```

A.3. Procesamiento ETL Diario

Archivo: airports_aggregation_daily.py

Script desarrollado en PySpark para AWS Glue. Este trabajo ETL lee los datos crudos desde la zona raw de S3, estandariza el esquema, convierte el formato y realiza una agregación de métricas agrupadas por día y país. El resultado se escribe en la zona processed.

```
import sys
import logging
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.utils import getResolvedOptions
from pyspark.sql.functions import col, sum as spark_sum, count, avg, desc
from awsglue.dynamicframe import DynamicFrame

# Configuración de logs
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

def main():
    args = getResolvedOptions(sys.argv, ['database', 'table', 'output_path'])
    database = args['database']
    table = args['table']
    output_path = args['output_path']

    sc = SparkContext()
    glueContext = GlueContext(sc)

    # Leer datos (Tabla RAW)
    dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
        database=database,
        table_name=table
    )

    df = dynamic_frame.toDF()

    daily_df = df.groupBy("country") \
        .agg(
            spark_sum("links_count").alias("total_enlaces"),
            count("name").alias("cantidad_aeropuertos"),
            avg("links_count").alias("promedio_enlaces")
        ) \
```

```
.orderBy(desc("total_enlaces"))

# Escribir resultado
output_dynamic_frame = DynamicFrame.fromDF(daily_df, glueContext,
"output")

glueContext.write_dynamic_frame.from_options(
    frame=output_dynamic_frame,
    connection_type="s3",
    connection_options={"path": output_path},
    format="parquet",
    format_options={"compression": "snappy"}
)

logger.info("Job finalizado correctamente.")

if __name__ == "__main__":
    main()
```

A.4. Procesamiento ETL Mensual

Archivo: airports_aggregation_monthly.py

Variación del script de procesamiento anterior diseñado para generar métricas consolidadas a nivel mensual. Realiza agregaciones de mayor nivel para facilitar el análisis de tendencias a largo plazo, optimizando el almacenamiento mediante particionado por fecha.

```
import sys
import logging
from pyspark.context import SparkContext
from aws glue.context import GlueContext
from aws glue.utils import getResolvedOptions
from pyspark.sql.functions import col, sum as spark_sum, count, avg,
substring, desc
from aws glue.dynamicframe import DynamicFrame

# Configuración de logs
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s
- %(message)s')
logger = logging.getLogger(__name__)

def main():
    args = getResolvedOptions(sys.argv, ['database', 'table',
'output_path'])
    database = args['database']
    table = args['table']
    output_path = args['output_path']

    sc = SparkContext()
    glueContext = GlueContext(sc)

    # Leer datos
    dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
        database=database,
        table_name=table
    )
    df = dynamic_frame.toDF()

    df = df.withColumn("month", substring(col("processed_at"), 1, 7))

    # Agregación Mensual
    monthly_df = df.groupBy("month", "country") \
        .agg(
```

```
        spark_sum("links_count").alias("total_enlaces"),
        count("name").alias("cantidad_aeropuertos"),
        avg("links_count").alias("promedio_enlaces")
    ) \
    .orderBy("month", desc("total_enlaces"))

output_dynamic_frame = DynamicFrame.fromDF(monthly_df, glueContext,
"output")

glueContext.write_dynamic_frame.from_options(
    frame=output_dynamic_frame,
    connection_type="s3",
    connection_options={"path": output_path},
    format="parquet",
    format_options={"compression": "snappy"}
)

logger.info("Agregación mensual completada.")

if __name__ == "__main__":
    main()
```

A.5. Script de Despliegue de Infraestructura (AWS CLI)

Archivo: scriptDML.sh

Este archivo recopila la secuencia de comandos ejecutados en la terminal (PowerShell) para el aprovisionamiento automatizado de los recursos en AWS. Incluye las instrucciones AWS CLI para:

- La creación del bucket S3 y la estructura de carpetas.
- El despliegue del flujo de datos en Kinesis Data Streams.
- La configuración del sistema de entrega en Kinesis Data Firehose.
- El despliegue y ejecución de los AWS Glue Crawlers para la catalogación de datos.

```
zip firehoseDML.zip firehoseDML.py

$env:AWS_REGION="us-east-1"
$env:ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
$env:BUCKET_NAME="cn-p2-airports-${env:ACCOUNT_ID}"
$env:ROLE_ARN=$(aws iam get-role --role-name LabRole --query 'Role.Arn'
--output text)

# Crear el bucket
aws s3 mb s3://${env:BUCKET_NAME}

# Crear carpetas (objetos vacíos con / al final)
aws s3api put-object --bucket $env:BUCKET_NAME --key raw/
aws s3api put-object --bucket $env:BUCKET_NAME --key processed/
aws s3api put-object --bucket $env:BUCKET_NAME --key config/
aws s3api put-object --bucket $env:BUCKET_NAME --key scripts/
aws s3api put-object --bucket $env:BUCKET_NAME --key queries/
aws s3api put-object --bucket $env:BUCKET_NAME --key errors/
aws kinesis create-stream --stream-name airports-stream --shard-count 1

# Lambda

aws lambda create-function --function-name airports-firehose-lambda
--runtime python3.12 --role $env:ROLE_ARN --handler
firehoseDML.lambda_handler --zip-file fileb://firehoseDML.zip --timeout 60
--memory-size 128
```

```

$env:LAMBDA_ARN=(aws lambda get-function --function-name
airports-firehose-lambda --query 'Configuration.FunctionArn' --output text)

# Firehose
aws firehose create-delivery-stream --delivery-stream-name
airports-delivery-stream --delivery-stream-type KinesisStreamAsSource
--kinesis-stream-source-configuration ("KinesisStreamARN=arn:aws:kinesis:" +
$env:AWS_REGION + ":" + $env:ACCOUNT_ID + ":stream/airports-stream,RoleARN="
+ $env:ROLE_ARN) --extended-s3-destination-configuration
('{\"BucketARN\": \"arn:aws:s3::' + $env:BUCKET_NAME + '\", \"RoleARN\": \"' +
$env:ROLE_ARN +
'\", \"Prefix\": \"raw/\", \"ErrorOutputPrefix\": \"errors/\", \"BufferingHints\
\": {\"SizeInMBs\": 1, \"IntervalInSeconds\": 60}, \"ProcessingConfiguration\": {\"E
nabled\": true, \"Processors\": [{\"Type\": \"Lambda\", \"Parameters\": [{\"Parame
terName\": \"LambdaArn\", \"ParameterValue\": \"' + $env:LAMBDA_ARN +
'\", {\"ParameterName\": \"BufferSizeInMBs\", \"ParameterValue\": \"1\"}, {\"Par
ameterName\": \"BufferIntervalInSeconds\", \"ParameterValue\": \"60\"}]}]}')

#Base de datos
aws glue create-database --database-input '{\"Name\": \"airports_db\"}'

#Crawler
aws glue create-crawler --name airports-raw-crawler --role $env:ROLE_ARN
--database-name airports_db --targets ('{\"S3Targets\": [{\"Path\": \"s3://' +
$env:BUCKET_NAME + '/raw/\"}]}')
aws glue create-crawler --name airports-processed-crawler --role
$env:ROLE_ARN --database-name airports_db --targets ('{\"S3Targets\":
[{\"Path\": \"s3://' + $env:BUCKET_NAME + '/processed/airports_daily/\"},
{\"Path\": \"s3://' + $env:BUCKET_NAME +
'/processed/airports_monthly/\"}]}')

aws glue start-crawler --name airports-processed-crawler
aws glue start-crawler --name airports-raw-crawler

#Script para crear el Job

aws s3 cp airports_aggregation_daily.py s3://$env:BUCKET_NAME/scripts/
aws s3 cp airports_aggregation_monthly.py s3://$env:BUCKET_NAME/scripts/

$env:DATABASE="airports_db"
$env:TABLE_INPUT="raw"
#Crear el Job
aws glue create-job --name airports-daily-aggregation --role $env:ROLE_ARN
--glue-version "4.0" --number-of-workers 2 --worker-type "G.1X" --command
('{\"Name\": \"glueetl\", \"ScriptLocation\": \"s3://' + $env:BUCKET_NAME +
'/scripts/airports_aggregation_daily.py\", \"PythonVersion\": \"3\"}')
--default-arguments ('{\"--database\": \"' + $env:DATABASE + '\",
\"--table\": \"' + $env:TABLE_INPUT + '\", \"--output_path\": \"s3://' +

```



```

$env:BUCKET_NAME + '/processed/airports_daily/',
"--enable-continuous-cloudwatch-log": "true"'))
aws glue create-job --name airports-monthly-aggregation --role $env:ROLE_ARN
--glue-version "4.0" --number-of-workers 2 --worker-type "G.1X" --command
('{\"Name\": \"glueetl\", \"ScriptLocation\": \"s3://' + $env:BUCKET_NAME +
'/scripts/airports_aggregation_monthly.py\", \"PythonVersion\": \"3\"}')
--default-arguments ('{\"--database\": \"' + $env:DATABASE + '\",
"--table\": \"' + $env:TABLE + '\", \"--output_path\": \"s3://' +
$env:BUCKET_NAME + '/processed/airports_monthly/',
"--enable-continuous-cloudwatch-log\": \"true\"}'))

aws glue start-job-run --job-name airports-daily-aggregation
aws glue start-job-run --job-name airports-monthly-aggregation

```

Anexo B: Reporte de Costes

Archivo: Estimacion_Costes_AWS_DML.pdf

Se adjunta a continuación el desglose detallado de la estimación de costes generado mediante AWS Pricing Calculator, correspondiente al escenario de producción descrito en el apartado 6 de esta memoria.

30/12/25, 19:38

My Estimate - Calculadora de precios de AWS



Póngase en contacto con su representante de AWS: [Comuníquese con el departamento de ventas](#)

Exportar fecha: 30/12/2025

Idioma: Español

URL estimada: <https://calculator.aws/#/estimate?id=55c9016ad208f32155b12eeebfc9ce6183e82f1c>

Resumen de la estimación

Costo inicial	Costo mensual	Costo total de 12 months
0,00 USD	56,06 USD	672,72 USD
		Incluye el costo inicial

Estimación detallada

Nombre	Grupo	Región	Costo inicial	Costo mensual
Amazon Kinesis Data Streams	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	22,64 USD

Estado: -**Descripción:**

Resumen de la configuración: Duración de la retención de datos (1 días), Número de registros de base de referencia (20 por segundo), Número máximo de registros (20 por segundo)

Amazon Data firehose	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	7,27 USD
----------------------	--------------------------------	-----------------------	----------	----------

Estado: -**Descripción:**

Resumen de la configuración: Tipo de origen (Secuencia de datos Direct PUT o Kinesis), Partición dinámica (complemento) (Desactivado), Proporción promedio de datos procesados con respecto a la VPC frente a los datos incorporados (1.3), Unidades de registros de datos (número exacto), Tamaño del registro (5 KB), Conversión de formato de datos (opcional) (Desactivado), Número de registros para la incorporación de datos (20 por segundo)

30/12/25, 19:38

My Estimate - Calculadora de precios de AWS

Amazon Simple Storage Service (S3)	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	7,44 USD
---	--------------------------------	-----------------------	----------	----------

Estado: -**Descripción:**

Resumen de la configuración: Almacenamiento de S3 Estándar (300 GB por mes), Solicitudes PUT, COPY, POST y LIST a S3 Standard (100000), Solicitudes GET, SELECT y todas las demás desde S3 Estándar (10000), Datos devueltos por S3 Select (50 GB por mes), Datos escaneados por S3 Select (0 GB por mes)

AWS Glue	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	17,60 USD
-----------------	--------------------------------	-----------------------	----------	-----------

Estado: -**Descripción:**

Resumen de la configuración: Número de DPU para el trabajo Apache Spark (4), Número de DPU para el trabajo Shell de Python (0.0625)

AWS Lambda	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	0,82 USD
-------------------	--------------------------------	-----------------------	----------	----------

Estado: -**Descripción:**

Resumen de la configuración: Arquitectura (x86), Cantidad de almacenamiento efímero asignado (512 MB), Arquitectura (x86), Modo de invocación (En búfer), Cantidad de solicitudes (2000000 por mes)

Amazon Athena	No se ha aplicado ningún grupo	US East (N. Virginia)	0,00 USD	0,29 USD
----------------------	--------------------------------	-----------------------	----------	----------

Estado: -**Descripción:**

Resumen de la configuración: Número total de consultas (1 por día), Cantidad de datos analizados por consulta (2 GB)

Reconocimiento

La Calculadora de precios de AWS proporciona únicamente una estimación de sus tarifas de AWS y no incluye los impuestos que puedan aplicarse. El valor real de sus tarifas depende de una serie de factores, entre los que se incluye su uso real de AWS. [Obtener más información](#)

Anexo C: Uso de la Inteligencia Artificial

Como se menciona en el proyecto docente de la asignatura, en este apartado se detalla el uso que se le ha dado a la Inteligencia Artificial (IA) como herramienta de apoyo para la realización de este trabajo.

En este proyecto, la IA se ha utilizado como una herramienta de asistencia en los siguientes apartados:

- Se empleó para diagnosticar mensajes de error en la consola de AWS, específicamente relacionados con la catalogación de tablas en AWS Glue y problemas de permisos o configuración en Amazon Athena.
- Sirvió como asistente para validar la sintaxis de los scripts de Python y las consultas SQL, asegurando la correcta transformación de los datos.
- Se utilizó para interpretar correctamente los parámetros técnicos de la AWS Pricing Calculator y definir un escenario de producción realista para el presupuesto.
- La IA fue una herramienta de apoyo en la redacción de esta memoria, ayudando a estructurar el contenido (especialmente los anexos) y generar referencias en formato APA.

Cabe destacar que todas las decisiones de diseño, la ejecución de los comandos y la verificación final de los resultados fueron realizadas por el alumno.

Anexo D: Enlace a Github

https://github.com/danimoreno73/CN_P2.git